

README

README • рисуем схемы вместе с LLM

Иногда хочется получить от LLM не буквы и не код, а **схемку**:

- «Сделай схему взаимодействия модулей в этой программе»
- «Нарисуй служебную иерархию в компании»
- «Покажи, как информация доходит от сервера в приложение»
- «Визуализируй, как связаны основные понятия этого документа»
- «Объясни в одной картинке работу алгоритма»

И результат должен быть не столько красивым, сколько точным:



Но потом хочется быстренько править эту схемку своими руками.

А лучше всего — буквами^[1].

<> Решено: будем только писать

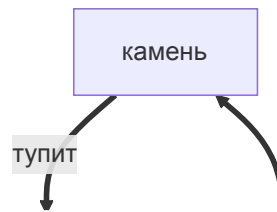
Эта идея — мы с LLM описываем связи, а «простой компьютер» их рисует — и воплощена в технологии^[2] [Mermaid](#).

Вот такие буквы...

...становятся картинкой

```
flowchart TB
    камень
    ножницы
    бумага

    камень ==> |тупит| ножницы
    ножницы ==> |режут| бумага
    бумага ==> |оборачивает| камень
```



💡 Да, это язык

💡 Нет, это не язык программирования

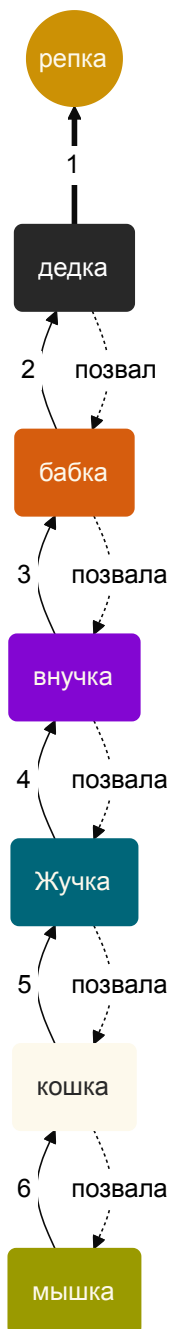
Какой промпт задать для начала

Собери мне в разметке Mermaid схему, иллюстрирующую отрисовку HTML-страницы в браузере. Покажи такие элементы: JavaScript, HTML, CSS, браузер, запрос пользователя, сервер. Называй их либо официальными названиями, либо по русски. К стрелкам сделай осмысленные надписи. Элементы пусть выглядят как прямоугольники со скруглёнными уголочками. И пусть всё выглядит так, будто нарисовано и раскрашено от руки. Результат я вставлю в Mermaid Editor и сразу захочу увидеть картинку.

Где дальше пробовать руками

- [Mermaid Editor](#) — копипаст слева, и сразу же картинка справа
- [Notion](#) и [Obsidian](#) — там не так удобно, но зато схема сразу в документе
- [Github](#) — редактируйте Markdown-файлы прямо через веб-интерфейс в репозитории

Чему стоит научиться



1. **Самое простое**, что позволяет Mermaid — это быстренько [набросать простую схему](#). Что у нас есть, что с чем связано, как что называется и что из чего следует. Вы записываете связи вроде `A ==> B` — точно как на уроках математики. А Mermaid их рисует. Больше связей — больше картинка. Где что лежит на этой картинке — забота технологии, не ваша.
2. Иногда **схема не так проста**. Например, одна из самых популярных технологических схем — так называемая диаграмма последовательностей — не очень удачно переводится на язык `A ==> B`. Но для неё [предусмотрен другой язык](#) — как и для многих других типов диаграмм. И снова то же самое: вы пишете буквами, что с чем и как связано — система сразу рисует картинку.
3. Если схема, которую вы нарисовали, **нужна не только вам** — стоит [поработать над её оформлением](#). Сделать так, чтобы в схеме было легко сориентироваться, чтобы на неё было приятно смотреть, и даже чтобы её было видно в других инструментах и системах. Со всем этим тоже помогает технология Mermaid.
4. Ну а если ваша задача большая и **схем нужно много** — тут уже нужны [решения высокого уровня](#). Почти программирование. И с этим Mermaid тоже поможет.

Какие ещё технологии встретятся

- **CSS** — система стилизации изображения; обязательна на всех уровнях после самого базового
- **JavaScript** — базовый стек; можно не вникать, если не разворачивать свой Mermaid-сервер
- **Markdown** — способ разметки текста; скорее всего, вы уже его знаете, даже если не видели названия
- **UML** — язык визуального оформления требований; скорее Mermaid полезен для него, чем наоборот
- **YAML** — система разметки для конфигурации конкретной диаграммы; обязательна на всех уровнях после самого базового

1. **LLM всегда генерит только буквы.** По определению. Даже если вам кажется, что результат — это что-то другое: эскизы интерфейса, схемы, сайты и т. п. Нет. Буквы. Чтобы превратить эти буквы в другие вещи, нужны разные технологии. ↩
2. **Технология** — это просто договорённость разных людей. Например, здесь все договорились, что из записи **A ==> B** машина может нарисовать картинку. И договорились, как именно это сделать. А нам остаётся только писать такие записи и получать картинки — там, где работает технология Mermaid. И не думать о том, как это сделано. ↩